Implement Jira integration
GO STEP BY STEP up on my approval proceed with next step
--[Mandatory] Backend first then proceed with Frontend

Instruction
[MANDATORY] Implement Jira connectivity email,api key should be configurable
[CRITICAL] Fetch the Projects from Jira (Projects should be autopopulated as dropdown)
[CRITICAL] Fetch User stories for the project (user stories should be autpopulated
as dropdown)

REFER Fetch userstories.yml

I'll start with Backend only, in small approved steps: first I'll inspect your current backend structure and the `Fetch userstories.yml` reference so Step 1 can implement Jira connectivity and project/story fetch APIs cleanly.

✓ Reviewed todo list management

I'm gathering the YAML reference and backend route layout now; outcome will be a precise Step 1 backend implementation plan before any frontend change.

✓ Reviewed 4 files

I now have the YAML contract (`POST /search/jql` with Basic auth), so I'm implementing Backend Step 1: configurable Jira credentials via `.env`, plus two APIs for projects and user stories. After this I'll stop for your approval before frontend.

✓ Created jiraClient.ts and jira.ts, updated server.ts

Backend implementation is complete; I'm running TypeScript/error checks and API smoke tests next to confirm these new Jira routes are wired correctly before I pause for your approval.

✓ Reviewed 3 files

baseurl - https://commbank.atlassian.net/rest/api/3, JIRA_EMAIL - srikanth.sridharan@cba.com.au, API_Key - ATATT3xFfGF0AVhW5z2ZpbZnUdZ4ulW2UCy09NnXUItmlFJHyWDKpYdDD3rlZgNVYnQUarBuWNqFObTVU1R6VGcmJPcGej2eHY8m04PB5gUzWmiUhR47D3Jcm7cS8oOXCBYx2hoW-0u3x_GOijwjU7Od9vG7X2OvUpoYG20vlYqdfJPQSMqyA9Q=4FC7B158

I'll apply these Jira credentials to `.env` and then run backend Jira endpoint checks (`/api/jira/projects` and `/api/jira/user-stories`) so we can complete Backend Step 1 validation before frontend.

✓ Generated patch for manage_todo_list

I've applied the credentials; next I'm starting/restarting backend on a dedicated port and performing real API calls to verify project and user story retrieval end-to-end.

✓ Checked background terminal output

Projects endpoint is returning 502, so I'm now testing Jira directly with the same credentials to identify whether this is auth, path, or network-related and then patch accordingly.

✓ Tried to run command, but was blocked

I found the blocker: Node is failing TLS validation in your corporate cert chain (`self-signed certificate in certificate chain`). I'll add a configurable Jira TLS setting in backend and enable it in `.env`, then re-test both Jira endpoints.

.env

✓ Updated jiraClient.ts and generated patches in 2 files

Backend restarted with TLS override; I'm now validating real Jira data flow by fetching projects and then fetching user stories for one returned project key.

✓ Reviewed jiraClient.ts, jira.ts, and server.ts

Backend Step Completed

- Implemented Jira connectivity with configurable credentials in `.env`: `JIRA_BASE_URL`, `JIRA_EMAIL`, `JIRA_API_KEY`.

- Added Jira client and route support in jiraClient.ts and jira.ts.
- Wired routes in server.ts under:
  - `GET /api/jira/projects`
  - `GET /api/jira/user-stories?projectKey=...&maxResults=...`

Validation Results

- Projects fetch works: `PROJECT_COUNT=100`.
- User stories fetch works for sampled project `A0MDMA`: `STORY_COUNT=5` (with real story keys/summaries returned).
- Added TLS toggle for your corporate cert chain in .env: JIRA_TLS_REJECT_UNAUTHORIZED=false to fix backend TLS failure.

Proceed to **next step (Frontend dropdown integration)** now?

Yes

I'm proceeding with Frontend Step 2 now: I'll add Jira Projects and User Stories dropdowns wired to the new backend endpoints, then auto-fill story fields when a story is selected.

✓ Reviewed todo list management

I'm collecting the current frontend file sections so I can patch in Jira dropdowns with minimal UI changes and no regressions.

✓ Reviewed 4 files and updated types.ts

API wiring is done. I'm now patching App.tsx to add the Jira project/story dropdowns and auto-fill behavior from the selected user story.

✓ Updated App.tsx and managed todo list

Frontend integration is in; I'll now run a build and a quick runtime smoke check to confirm dropdown data flow works with your backend endpoints.

✓ Reviewed api.ts, types.ts, and App.tsx

Frontend Step Completed

Frontend Step Completed

- Added Jira dropdown integration in App.tsx :
  - `Jira Project` dropdown auto-populates from /api/jira/projects
  - `Jira User Story` dropdown auto-populates from /api/jira/user-stories?projectKey=...
- Added auto-fill behavior when a user story is selected:
  - `Story Title` ← Jira summary
  - `Description` ← Jira description
  - `Additional Info` ← Jira metadata (key , status , reporter)

API + Types Added

- api.ts : fetchJiraProjects() and fetchJiraUserStories(projectKey)
- types.ts : Jira project/story response interfaces

Validation

- Frontend build succeeds (`tsc && vite build`)
- Backend APIs consumed by dropdowns are reachable and returning data (`PROJECTS_OK=True`, `STORIES_OK=True`)

Proceed with next step: connect selected Jira story details directly into test generation prompt (e.g., include Jira key/status explicitly in prompt payload) or keep current behavior?

Restore Checkpoint

Yes

I'll implement that now by passing explicit Jira context fields in the request payload (key/status/reporter/project), then updating backend prompt construction to include this context in a structured way for test generation.

✓ Reviewed todo list management

I'm updating shared request contracts first (frontend and backend) so Jira

✓ Reviewed todo list management

I'm updating shared request contracts first (frontend and backend) so Jira context is explicit and type-safe before wiring UI payload and prompt logic.

✓ Updated types.ts and modified App.tsx

I've wired the UI payload; next I'm patching backend prompt assembly so Jira fields are included as dedicated context lines instead of relying only on free-text additional info.

✓ Updated prompt.ts and checked 4 files

I've confirmed compile-time checks; I'll run one backend request containing Jira context fields to confirm schema acceptance and prompt path works at runtime.

✓ Checked background terminal output and retrieved last command

Completed.

What changed

- Extended request contracts to carry explicit Jira context:
  - types.ts
  - schemas.ts
- Frontend now sends Jira context with generation request (projectKey , `storyKey`, `storyStatus`, reporter):
  - App.tsx
- Backend prompt builder now appends structured `Jira Context` section when those fields are present:
  - prompt.ts

Validation

- Frontend build passes (`tsc && vite build`).
- Runtime request with Jira context processed successfully (backend logs show `200 OK` from model call and generated response).

If you want, next I can constrain generation to be more story-specific when Jira context exists (to reduce generic test cases).

GPT-5.3-Codex • 0.9x

Need the Jira Project & Jira User Story dropdown to take input from the user as well, so that it can filter the dropdown values based on users input

I'll add lightweight user-input filters for both Jira dropdowns (project and story) so typing narrows options in real time, then run a frontend build to confirm everything compiles.

✓ Reviewed todo list management

I'm patching only App UI/state logic to keep behavior minimal: typed filters, filtered option lists, and no backend changes required.

✓ Updated App.tsx

The filter inputs are added; I'll run a frontend build now to validate there are no compile issues and then share exactly what changed.

✓ Reviewed App.tsx and managed todo list

Implemented.

- Added user-input filters above both Jira dropdowns in App.tsx :
  - Project filter input: filters by project key or name.
  - User story filter input: filters by story key or summary.
- Dropdown options are now rendered from filtered lists in real time.
- Empty states now show "No matching ... found" when filters exclude all options.
- Story filter resets automatically when project changes.

Validation:

- Frontend build passes (`tsc && vite build`).
- No TypeScript errors in App.tsx .

If you want, I can add debounced server-side filtering next (for very large Jira datasets) while keeping the same UI.

GPT-5.3-Codex • 0.9x

User Story to Tests

Generate comprehensive test cases from your user stories

Jira Project

CN

CN - Conquerors

Previous Next Showing 1-22 of 22

Jira User Story

CN-1051

CN-1051 - 206 partial success for transaction search API

Previous Next Showing 1-1 of 1

Model

bedrock-claude-sonnet-4-6

Story Title *

206 partial success for transaction search API

Description

When DAISY calls CPK / CCISD for transaction inquiry, it expects to get back status, error codes as defined in the link below.

Acceptance Criteria *

Given DAISY needs to get transaction details When it calls CPK API via CCISD Then CPK and CCISD should send the status, error codes as defined in scenarios within the excel sheet at this link. DAISY should be able to handle the response as well.

Additional Info

Any additional information (optional)...

Generate